

PROJECT
ON

REMOTE REWORK UPDATE SYSTEM

AIM OF THE PROJECT:

To develop a communication system enabling communication from the reworker to multiple debugging engineers

WORKING PRINCIPLES:

The communication system is designed such that, on the master side, a button switch **S0** is used to select which receiver will receive the message. Another switch, **S1**, is used to send different messages to a single receiver. The messages sent to the receiver are as follows:

- **Blue LED** indicates that the reworker is free to work on a component.
- **Red LED** indicates that the reworker has been assigned a component.
- **Yellow LED** indicates that work is in progress on a component.
- **Green LED** indicates that the reworker has completed the required work on a component, and a **buzzer** will be **activated** at the receiver end for 5 seconds.

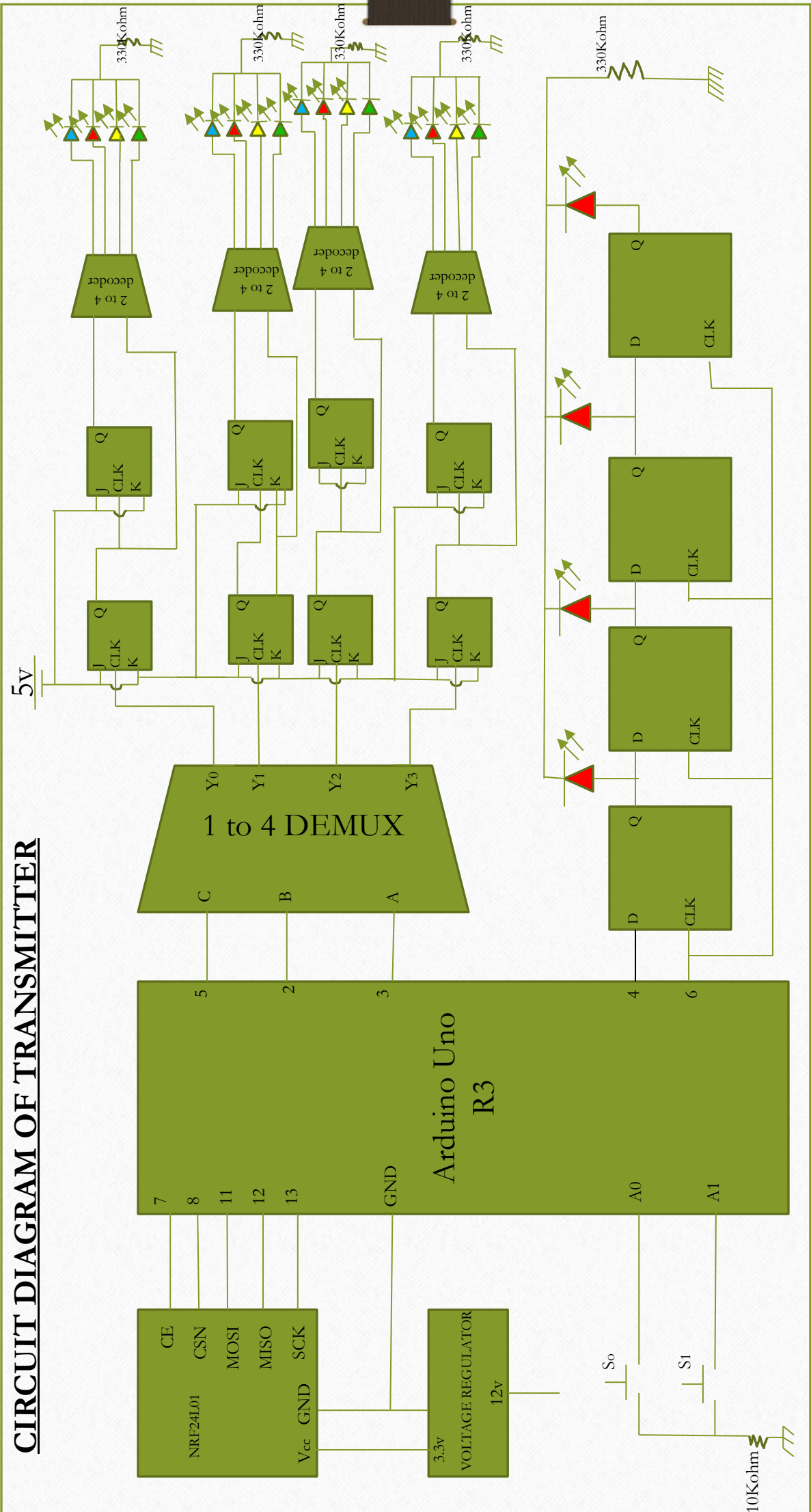
COMPONENTS USED AT TRANSMITTER END

- Arduino Uno R3
- 12V to 3.3V voltage regulator (made by two diodes, LD33CV, heat sink, resistors(22ohms) and capacitors)
- NRF24L01 (having PA and LNA)module
- D flipflops(IC7474)
- 2 to 4 decoders (IC74139)
- 1:4 demultiplexer(IC74155)
- JK flipflop(IC7476)
- LEDs(blue, red, yellow and green)
- Breadboard and jumperwires

COMPONENTS USED AT RECIEVER END

- Arduino Uno R3
- 12V to 3.3V voltage regulator (made by two diodes, LD33CV, heat sink, resistors(22ohms) and capacitors)
- NRF24L01 (having PA and LNA)module
- LEDS(blue, red, yellow and green) and an active buzzer
- Breadboard and jumperwires

CIRCUIT DIAGRAM OF TRANSMITTER



ARDUINO CODE FOR TRANSMITTER SIDE

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

RF24 radio(7, 8); // CE, CSN

// 5-byte addresses for 4 receivers
const byte addresses[4][6] = {
  "R0001", // Receiver 0
  "R0002", // Receiver 1
  "R0003", // Receiver 2
  "R0004" // Receiver 3
};

// Inputs
const int patternShiftButton = A0; // Change target receiver
const int pulseButton = A1; // Send signal

// Outputs
const int pin2 = 2; // Logic out
const int pin3 = 3; // Logic out
const int dataPin = 4; // Data for shift
const int pulsePin = 5; // Pulse trigger
const int clockPin = 6; // Clock for shift

// State variables
int patternState = 0;
bool lastPatternBtn = HIGH;
bool lastPulseBtn = HIGH;

int shiftData[4] = {1, 0, 0, 0}; // Custom data for each receiver
int currentBit = 0;

void setup() {
  Serial.begin(9600);
  radio.begin();
  radio.setPALevel(RF24_PA_MAX);
  radio.setDataRate(RF24_1MBPS);

  radio.stopListening();

  pinMode(patternShiftButton, INPUT_PULLUP);
  pinMode(pulseButton, INPUT_PULLUP);

  pinMode(pin2, OUTPUT);
  pinMode(pin3, OUTPUT);
  pinMode(dataPin, OUTPUT);
  pinMode(clockPin, OUTPUT);
  pinMode(pulsePin, OUTPUT);

  digitalWrite(pulsePin, LOW);
  digitalWrite(dataPin, LOW);
  digitalWrite(clockPin, LOW);
}

void loop() {
  // --- Change receiver address ---
  bool currentPatternBtn = digitalRead(patternShiftButton);
  if (currentPatternBtn == LOW && lastPatternBtn == HIGH) {
    patternState = (patternState + 1) % 4;

    // Indicate address on logic outputs
    switch (patternState) {
      case 0: digitalWrite(pin2, LOW); digitalWrite(pin3, LOW); break;
      case 1: digitalWrite(pin2, LOW); digitalWrite(pin3, HIGH); break;
      case 2: digitalWrite(pin2, HIGH); digitalWrite(pin3, LOW); break;
      case 3: digitalWrite(pin2, HIGH); digitalWrite(pin3, HIGH); break;
    }

    // Simulate one bit shift
    digitalWrite(dataPin, shiftData[currentBit]);
    digitalWrite(clockPin, HIGH);
    delay(1);
    digitalWrite(clockPin, LOW);
    currentBit = (currentBit + 1) % 4;

    delay(200); // debounce
  }

  lastPatternBtn = currentPatternBtn;

  // --- Send signal to selected receiver ---
  bool currentPulseBtn = digitalRead(pulseButton);
  if (currentPulseBtn == LOW && lastPulseBtn == HIGH) {
    // Pulse signal
    digitalWrite(pulsePin, HIGH);
    delay(100);
    digitalWrite(pulsePin, LOW);

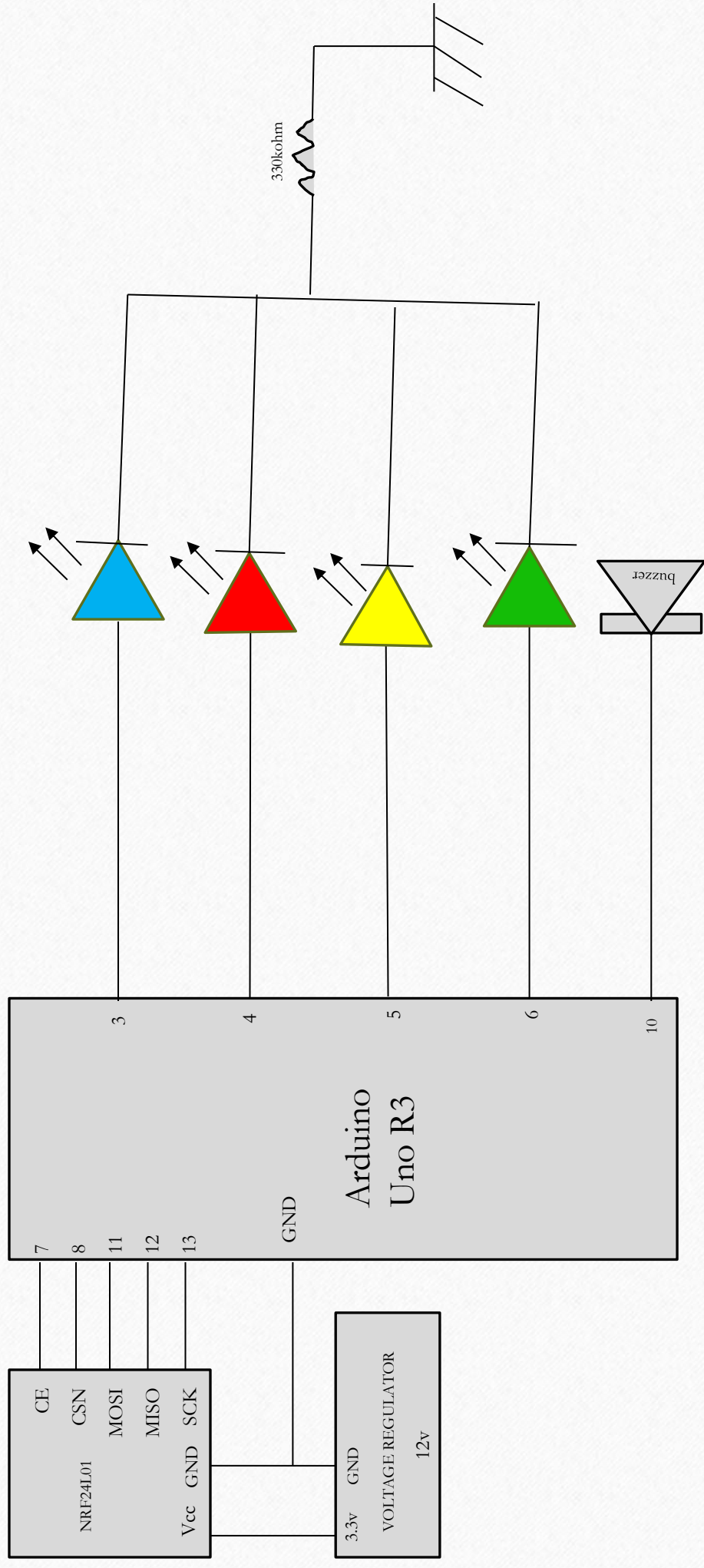
    // Transmit to selected receiver
    radio.openWritingPipe(addresses[patternState]);
    bool success = radio.write(&patternState, sizeof(patternState));

    Serial.print("Sent index ");
    Serial.print(patternState);
    Serial.print(" to ");
    Serial.print((char*)addresses[patternState]);
    Serial.println(success ? " ✓ OK" : " ✗ Failed");

    delay(200); // debounce
  }

  lastPulseBtn = currentPulseBtn;
}
```


SCHEMATIC DIAGRAM OF RECIEVER



ARDUINO CODE FOR RECIEVER SIDE

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

RF24 radio(7, 8); // CE, CSN

const byte thisAddress[6] = "R0001"; // Unique address for this receiver

// Output pins
const int ledPins[4] = {3, 4, 5, 6}; // LEDs: pin 3 to 6
const int buzzerPin = 10; // Buzzer: pin 10

int currentStep = 0; // Index to track which pin to activate (0 to 3)

void setup() {
  Serial.begin(9600);

  // Initialize LED pins and buzzer
  for (int i = 0; i < 4; i++) {
    pinMode(ledPins[i], OUTPUT);
    digitalWrite(ledPins[i], LOW);
  }

  pinMode(buzzerPin, OUTPUT);
  digitalWrite(buzzerPin, LOW);

  // Set up nRF24L01
  radio.begin();
  radio.setPALevel(RF24_PA_MAX);
  radio.setDataRate(RF24_1MBPS);
  radio.openReadingPipe(0, thisAddress);
  radio.startListening();

  Serial.println("Receiver R0001 is ready...");
}
```

```
void loop() {
  if (radio.available()) {
    int receivedIndex;
    radio.read(&receivedIndex, sizeof(receivedIndex));
    Serial.print("Received index: ");
    Serial.println(receivedIndex);

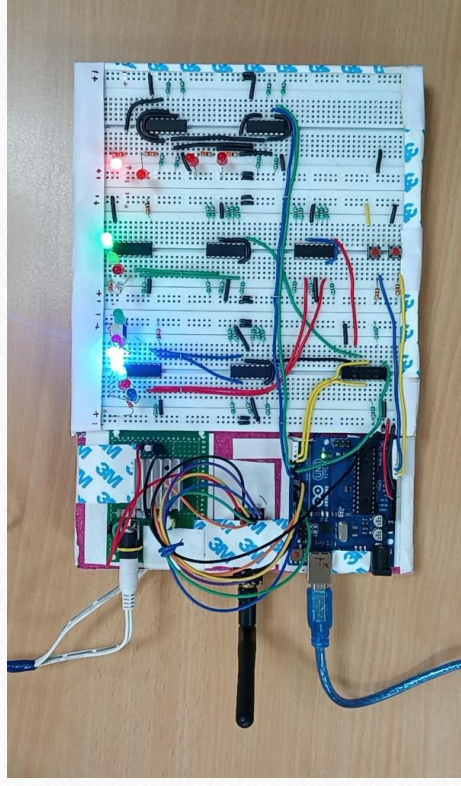
    if (receivedIndex == 0) {
      // Turn off all LEDs and buzzer
      for (int i = 0; i < 4; i++) {
        digitalWrite(ledPins[i], LOW);
      }
      digitalWrite(buzzerPin, LOW);

      // Turn ON the current LED
      digitalWrite(ledPins[currentStep], HIGH);
      Serial.print("Turning ON pin: ");
      Serial.println(ledPins[currentStep]);

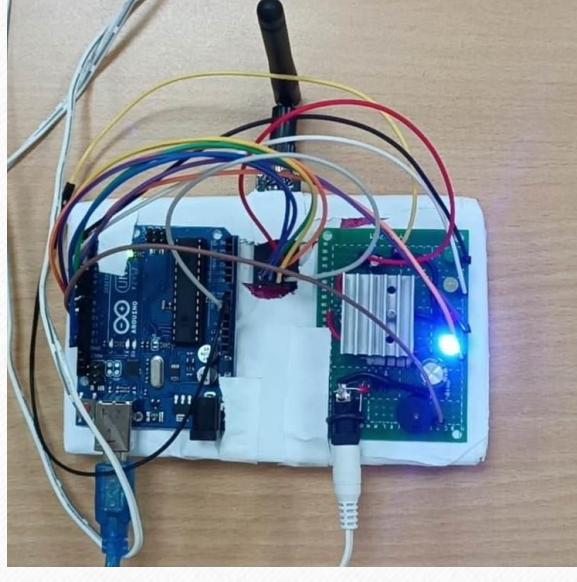
      // If the current pin is pin 3 (index 0), turn ON buzzer for 10 sec
      if (currentStep == 0) {
        digitalWrite(buzzerPin, HIGH);
        Serial.println("Buzzer ON for 5 seconds");
        delay(5000); // Blocking delay for buzzer
        digitalWrite(buzzerPin, LOW);
        Serial.println("Buzzer OFF");
      }

      // Move to the next step (0 → 1 → 2 → 3 → 0 ...)
      currentStep = (currentStep + 1) % 4;
    }
  }
}
```


PICTURE OF TRANSMITTER AND RECEIVER

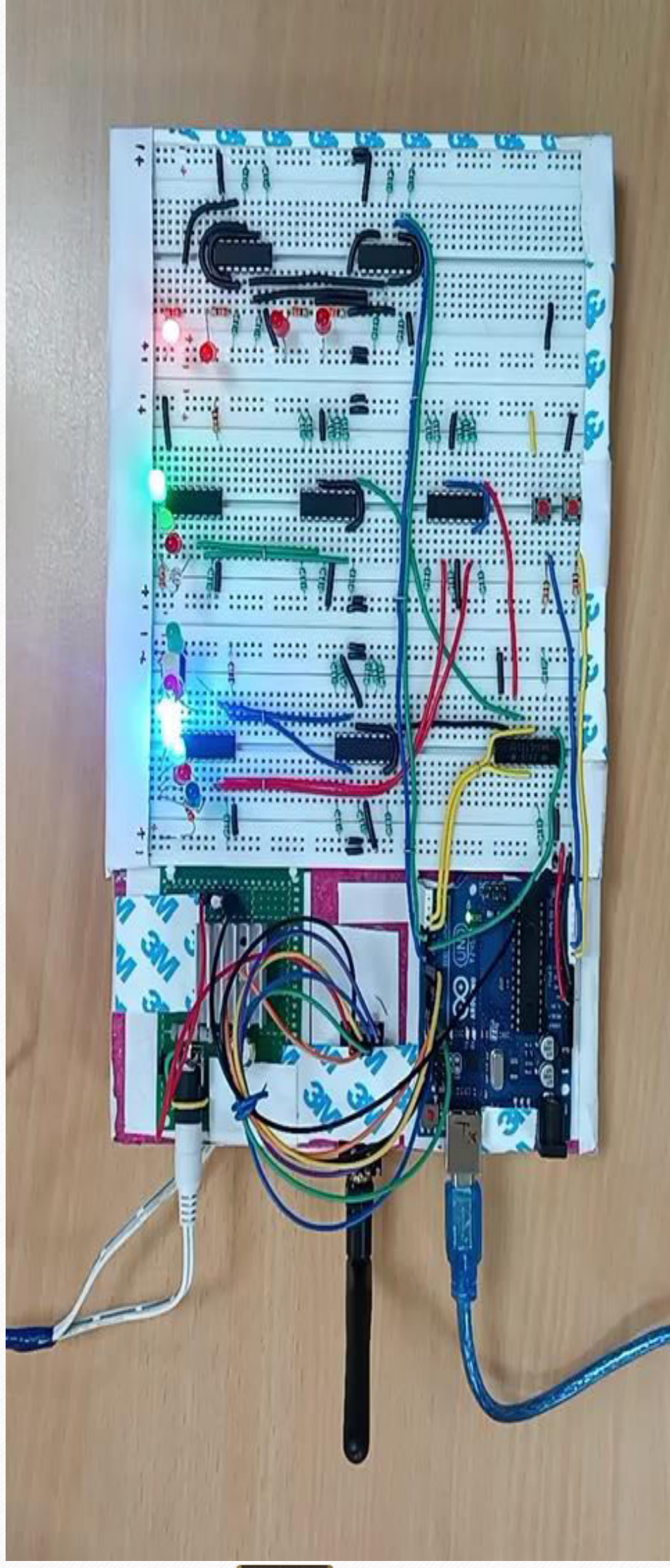


TRANSMITTER



RECEIVER

VIDEO DEMONSTRATION



THANK YOU